

Prague University of Economics and Business
Faculty of Informatics and Statistics



Deep Reinforcement Learning for the Card Game Dominion

BACHELOR THESIS

Study program: Data Analytics

Author: Vojtěch Kolomazník

Supervisor: doc. Ing. Ondřej Zamazal, Ph.D.

Prague, May 2025

Acknowledgements

I am grateful to my thesis supervisor doc. Ing. Ondřej Zamazal, Ph.D. for guiding me and helping me balance experimentation with academic rigour during the writing of this thesis.

I also want to thank my friend Adam, who sparked my interest in reinforcement learning and provided valuable feedback relating to my efforts.

Abstract

This thesis aims to propose an approach using deep reinforcement learning (deep RL) for the card game Dominion. Existing deep RL approaches are studied and compared, so that a deep RL agent can be implemented. After that, the thesis attempts to determine whether a properly trained agent can perform above the level of a simple heuristic, find out which adjustments allow the agent to perform better in a partially observable environment and study the choices made by the agent. Dominion is a deck-building card game, which presents many challenges to the RL community. It violates the Markov property because the state is not fully observable to the agent. Moreover, there is inherent stochasticity because the agent's deck gets shuffled between turns. Current academic research has attempted to apply deep Q-networks and policy gradients to solve the game with varying success. One of the biggest unanswered questions relates to the calculation of the rewards. Both Monte Carlo and temporal-difference methods yield sufficient performance in different scenarios. For the purpose of this thesis, several modifications were added to decrease the complexity of the problem. The selection of cards used in the game was frozen between games and the action phase of an agent's turn was handled with a heuristic. Only the buying phase was controlled directly by the agent. For this thesis, REINFORCE method relying on policy gradients and Monte Carlo returns displayed superior performance compared to deep Q-networks and actor-critic architectures using temporal-difference returns. It obtained a 95 percent win rate against Big Money heuristic and an 88 percent win rate against Smithy heuristic after only 20 thousand simulated games.

Keywords

reinforcement learning (RL), neural networks (NN), Q-learning, deep Q-networks (DQNs), policy gradients (PGs), REINFORCE, actor-critic (A2C), temporal-difference (TD), Monte Carlo (MC), Markov decision process (MDP), partially observable Markov decision process (POMDP)

Contents

Introduction	6
1 Reinforcement Learning	7
1.1 Example Of a Reinforcement Learning Problem	8
1.2 Traditional Approaches	8
1.2.1 Monte Carlo	9
1.2.2 Temporal-Difference	9
2 Deep Reinforcement Learning	11
2.1 Artificial Neural Networks	11
2.2 Deep Q-Networks	12
2.3 Policy Gradients	13
2.3.1 REINFORCE Algorithm	13
2.3.2 Actor-Critic Algorithm	14
2.4 Comparison	14
3 Deep Reinforcement Learning for the Card Game Dominion	16
3.1 The Card Game Dominion in Reinforcement Learning	16
3.2 Heuristics	17
3.3 Existing Approaches	18
3.4 Comparison	19
4 Design of Deep Reinforcement Learning approach	21
4.1 Thesis Goals and Research Questions	21
4.2 Methodology	22
4.3 Approach	22
4.3.1 Game Representation	22
4.3.2 Agent Design Choices	23
4.4 Implementation Details	24
4.5 Experiments and their results	28
4.5.1 DQN Agent	29
4.5.2 A2C Agent	31
4.5.3 REINFORCE Agent	33
4.5.4 Summary	34
4.6 Comparison To Other Approaches	37
Conclusion	39
References	41
A Source Code	45

List of abbreviations

RL reinforcement learning	i.i.d. independent and identically distributed
MDP Markov decision process	PG policy gradient
POMDP Partially observable Markov decision process	A2C (advantageous) actor-critic
MC Monte Carlo	tanh hyperbolic tangent
TD temporal-difference	ReLU rectified linear unit
SARSA state, action, reward, state, action	MLP multilayer perceptron
NN neural network	RQ research questions
DQN deep Q-network	

Introduction

Methods using deep reinforcement learning (deep RL) have surpassed human performance in many games in the last few years. Notable examples include AlphaZero achieving superhuman performance in games like Go and chess (Silver et al., 2018). Although the algorithm itself is impressive, these games follow the Markov property as they offer perfect information about the environment and do not contain any elements of stochasticity (Sutton & Barto, 2018, pp. 441–450).

The card game Dominion is a popular deck building card game that violates some of the Markov properties present in other reinforcement learning problems (Fernandez & Anderson, 2020), pushing it into the territory of partially observable Markov decision processes, also known as POMDPs. Although there have been previous attempts at training deep RL agents for this game, there is no comparison between the performance of various methods, and many papers approach the problem with the same assumptions that would be present in a classic MDP problem (see Section 3.2 for details).

This thesis aims to propose an approach using deep reinforcement learning for the card game Dominion. To implement and train deep RL agent, existing solutions are analyzed and compared. One goal is to find out whether a properly trained deep RL agent can perform above the level of a simple heuristic. The second goal is to find modifications of the agent that allow it to perform better in a partially observable environment. The last goal is to analyze the impact of specific choices made by a trained agent in the game.

The paper follows a methodology where existing solutions are analyzed and a deep RL agent is designed with the aim of achieving sufficient performance. The agent is implemented and trained in Python (Python Software Foundation, 2024). Once trained, it is compared to a heuristic bot based on selected benchmarks. If it fails to outperform the bot, the previous steps are repeated. If it achieves sufficient performance, quantitative and qualitative metrics are measured for comparison and understanding of its performance (see Section 4.2 for details).

Three agents were created and tested:

- A deep Q-network agent using Q-values as errors (see Section 4.5.1).
- A policy gradient actor-critic agent using temporal difference errors (see Section 4.5.2).
- A policy gradient REINFORCE agent using Monte Carlo errors (see Section 4.5.3).

Out of these, the last one proved to be the most successful at meeting the goals of this thesis. It outperformed the Big Money heuristic and achieved stable performance. A detailed evaluation of the agents against the goals of this thesis is available in Section 4.5.4.

1. Reinforcement Learning

At the most basic level, living organisms make decisions that allow them to exploit the environment that they inhabit. Certain sets of behaviors can bring rewards, which can be immediate; such as receiving nourishment or additional resources, or they can span over the long term; such as expanding the pool of available resources or proliferation of a species within an ecosystem (Darwin & Huxley, 2003).

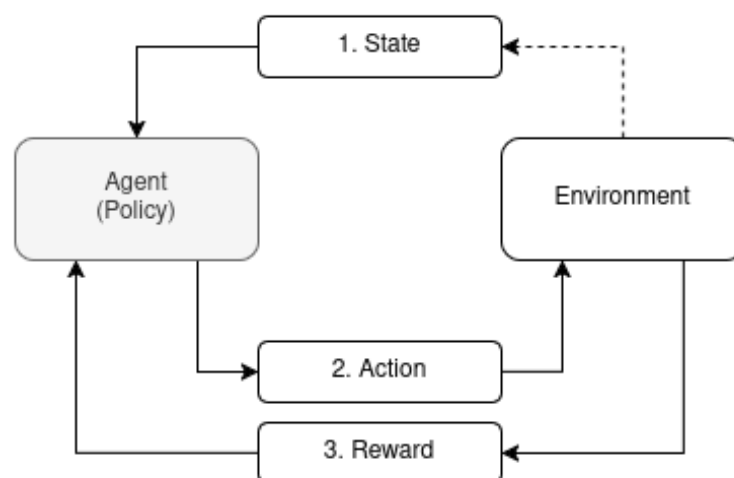
Within the scope of machine learning, the field of reinforcement learning (RL) emulates these behaviors to a certain extent. In place of an organism, an *agent* is used to describe an actor that makes decisions. The decisions that an agent makes are called *actions*. But in order for those decisions to have an effect, an interactive *environment* has to be present (Sutton & Barto, 2018, p. 2).

This environment can be as simple as balancing a pole on a cart (Geva & Sitte, 1993) or as complex as a simulated game on the Atari console (Mnih et al., 2013).

Much like in real life, an agent has limited information about the environment. This set of information is called the *state* (Sutton & Barto, 2018, p. 2). In reinforcement learning, the state is usually represented as a vector of n dimensions that contains important information about the environment. This can be any observation that is available to the agent at the time (Maillard et al., 2011). Lastly, the agent receives *rewards* that are used to guide future decisions. Both the state and the structure of the rewards are major contributors to the *policy*, which is a set of behaviors learned by the agent (Sutton & Barto, 2018, p. 6). Figure 1.1 provides an overview of these concepts.

Figure 1.1

Conceptual model of reinforcement learning. Adapted from basic usage of Gymnasium package (Farama Foundation, 2024); source: Author.



1.1 Example Of a Reinforcement Learning Problem

Let us consider a very simple case for reinforcement learning; the game of Tic-Tac-Toe. Although it can be solved with simple decision trees (Ho et al., 2023) such a solution would not necessarily be useful in a real-life situation. Contrary to the definition of an optimization problem, the other player cannot be expected to make perfect moves all the time (Sutton & Barto, 2018, p. 30). Moreover, a deterministic solution would suffer from the curse of dimensionality if the problem complexity were to increase (Bellman, 1966). For these reasons, reinforcement learning can offer an interesting alternative approach.

RL approach to a game of Tic-Tac-Toe would involve the following steps:

- The game state is formalized as a vector of n fields. For a 3x3 grid, this would entail 9 fields. Marks placed by the player could be represented with 1s and those placed by the enemy would be marked with -1s.
- A reward system needs to be created. The game can end in a win, draw or a loss. Special care should be taken as the structure of the reward strongly affects the success of the agent (Chapman et al., 2013).
- A learning approach should be selected. There are numerous approaches available and listing them all is outside the scope of this work. However, these are the relevant approaches that are mentioned later in the thesis:
 - The agent can calculate the quality of a state and aim for states with the highest quality value (Sutton & Barto, 2018, pp. 58-62). This approach favors exploitation over exploration, so care needs to be taken to prevent ending up in a local maxima (Tokic, 2010).
 - The agent can aim for a balanced policy, regardless of the intermediate reward (Sutton & Barto, 2018, pp. 58-62). This approach has the benefit of introducing stochasticity into the search (Williams, 1992). However, it is complex and can be more difficult to implement.

This simple example allows us to examine that reinforcement learning can be used to break down and model a relatively complex real-life problem so that a computer can understand it.

1.2 Traditional Approaches

Traditional RL methods can be split into many types according to the way the reward is attributed. This affects the way that experiences are collected. Some approaches can only work with complete episodes, whereas others are capable of learning in an environment that relies on continuous interaction. They are described in this chapter as *Monte Carlo* (MC) approaches and *temporal-difference* (TD) approaches respectively. A real world example suited to MC methods is a card game with a set amount of turns. A real world example suited for TD methods is a bot trying to survive in a computer game without a time limit.

However, there is a wide range of problems where both methods are applicable and there are hybrid methods that combine different aspect of the two. When deciding which approach to use, it should be noted that the approach can affect the convergence dynamics of an agent and lead to different results on the same problem (Sutton & Barto, 2018, pp. 115-116).

Connected to these approaches is the aspect of *on-policy* or *off-policy* learning. On-policy algorithms can only improve if the experiences contain choices made by the trained agent. Off-policy algorithms can learn from data that was generated during previous training or generated through random experiences. The policy requirement of the approach can affect the convergence dynamic of the agent. On-policy algorithms generally converge faster, but generating data for them is more difficult. Data for off-policy algorithms can be generated much faster, or even reused, but at some cost to the speed of convergence of the agent (Sutton & Barto, 2018, pp. 100-108).

1.2.1 Monte Carlo

Monte Carlo (MC) is a class of RL methods that learn only from completed episodes. They can estimate both the values of the state or the quality of the current policy. The reward is attributed backwards to states once the whole episode has ended. Over the course of many episodes, it should converge to the expected value (Sutton & Barto, 2018, pp. 81-82). The value of state, which the agent relies on to guide its decisions, is calculated as follows:

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)] \quad (1.1)$$

where the value of a state $V(S_t)$ is approximated by adding the difference of the final reward G_t and the current estimate $V(S_t)$ to the current estimate and adjusting it using the learning rate α (Sutton & Barto, 2018, p. 92).

MC does not require a model of the environment as it can learn a policy from pure interaction. Another advantage is that MC can focus on a small subset of states and learn efficiently from them by following a policy. However, MC methods require complete episodes to learn, which can be impossible under certain conditions, such as when working with continuous tasks (Sutton & Barto, 2018, pp. 115-116).

1.2.2 Temporal-Difference

Methods that use temporal-difference (TD) approach do not require complete episodes to learn. By *bootstrapping*, i.e. updating future estimates based on currently learned estimates, they can learn from transitions between states (Sutton & Barto, 2018, p. 118). This allows them to be applied to different kinds of problems, but are more sensitive to violations of the Markov property (Lapan, 2018, p. 118). The estimation of value of state is as follows:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] \quad (1.2)$$

where the value of state $V(S_t)$ is approximated by adding the immediate reward R_{t+1} and the discounted value of the next state $\gamma V(S_{t+1})$ and subtracting value of the current state $V(S_t)$. The result is modified by learning rate α and used to update the current estimate. TD methods can be faster to converge in some environments, and there are both on-policy and off-policy versions available (Sutton & Barto, 2018, p. 120). They are explained in the following sections as *SARSA* and *Q-Learning* respectively.

SARSA

SARSA, which stands for *state-action-reward-state-action*, offers a slight modification of the base temporal-difference method. It does not estimate the value of the state, like the base version of TD learning, but it estimates the value of an action (Sutton & Barto, 2018, p. 129). This is more useful in practice as it gives the agent feedback directly on which actions it should follow (Lapan, 2018, pp. 125-127). The equation for this estimation is as follows:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \quad (1.3)$$

Instead of calculating the value of state $V(S_t)$, the value of action $Q(S_t, A_t)$ is calculated. It is also called the Q-value. To approximate it, the reward R_{t+1} is added to the discounted Q-value of the next state $\gamma Q(S_{t+1}, A_{t+1})$ and the current estimate $Q(S_t, A_t)$ is subtracted. The result is modified by the learning rate α and used to update the current estimate. This method calculates the Q-value for the currently pursued policy, making it an on-policy algorithm (Sutton & Barto, 2018, pp. 129-131).

Q-Learning

Q-learning functions in a similar way to SARSA. Instead of calculating the value of the currently pursued policy, it calculates the value of the best available policy. This makes it easier to generate data for the agent as it can learn from past experiences, even if the agent previously followed a suboptimal policy (Sutton & Barto, 2018, pp. 131-132). The equation for estimating the Q-value is as follows:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)] \quad (1.4)$$

The only difference compared to SARSA is that the Q-value is calculated using the discounted value of the best available action $\gamma \max_a Q(S_{t+1}, a)$, which is used to update the Q-value of the current state $Q(S_t, A_t)$ (Sutton & Barto, 2018, p. 131). The details of implementing a Q-learning algorithm with neural networks are explored in Section 2.2.

2. Deep Reinforcement Learning

The previous chapter outlined how classical reinforcement learning attempts to make the best possible decision given an input state. When trying to implement this concept in practice, multiple issues become apparent. How can the best decision be found in a very large state space? A similar state should yield a similar decision, but if it has not been observed yet, how can an agent decide on the best approach?

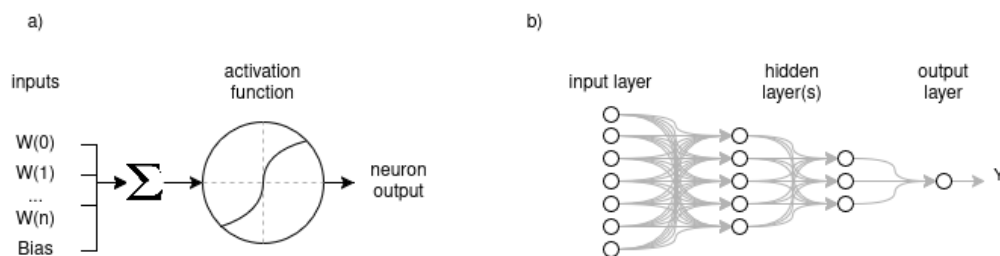
2.1 Artificial Neural Networks

Artificial neural networks, shortened to neural networks (NNs), help deal with these issues. NNs work by simulating the function of neurons found in the brain. Mathematically, the layers of NNs are made up of neurons, where each neuron represents a linear equation expressed as $y = mx + b$. Unlike linear models, NNs use activation functions such as the *sigmoid*, *hyperbolic tangent* (*tanh*) or *rectified linear unit* (*ReLU*), which allows them to approximate nonlinear functions (Goodfellow et al., 2016, pp. 193-195). Adding more neurons to a single layer allows the network to capture complex patterns. Adding more layers to a network allows it to extract its own features, bypassing the need for manual feature engineering of the input data (Goodfellow et al., 2016, p. 5).

A deep NN contains an input layer, one or more hidden layers and an output layer. Figure 2.1 displays the architecture of a deep NN, similar to the one used in Chapter 4. Part a) of the figure displays an individual neuron. It takes input of all input weights W_n and a bias term. The result neuron output is obtained by summing the weights and adding the bias term. The sum is passed through a tanh activation function, which limits the result to a value between $[-1, 1]$. The part b) displays a deep NN, also called a multilayer perceptron. As mentioned previously, each additional neuron in a layer helps it detect more complex patterns and each additional layer helps to abstract complex features (Goodfellow et al., 2016, pp. 168-169).

Figure 2.1

Schematic representation of a) a neuron, b) a deep neural network. Adapted from (Zanotti et al., 2019); source: Author.



In traditional RL, an agent would need to visit every state and possibly every action in order to judge their quality. A NN can create a good approximation of the underlying function given an appropriate amount of training data (Goodfellow et al., 2016, pp. 19-20). This makes it possible for the agent to generalize and judge the quality of a previously unseen state and action based on the patterns learned by the NN (Sutton & Barto, 2018, p. 195). Moreover, they become especially useful for reinforcement learning for the following reasons:

- NNs require a large amount of training data (Mnih et al., 2013). Unlike supervised machine learning, where samples have to be labeled, reinforcement learning can easily create new samples via a simulated game environment (Sutton & Barto, 2018, p. 91).
- NNs implicitly simplify the state of the environment down to the actions available for the agent (Mousavi et al., 2018).
- A well fitted NN can generalize well on previously unseen states (Goodfellow et al., 2016, p. 98).

Within the scope of reinforcement learning, a NN is added to the agent to help simplify and guide their decisions. In the previous chapter, two distinctions were made: agents can aim to maximize good states or they can focus on developing a good policy. Two techniques are explored, namely deep Q-networks and policy gradients.

2.2 Deep Q-Networks

Deep Q-networks (DQNs) are an extension of the Q-learning algorithm introduced in the previous chapter. They take advantage of the fact that the Q-value can be expressed via itself, which makes it possible to train with simple transitions (Lapan, 2018, p. 125). The Q-value approximation is as follows:

$$Q(s, a) = r_{s,a} + \gamma \max_{a' \in \mathcal{A}} Q(s', a') \quad (2.1)$$

where $r_{s,a}$ is the immediate reward, γ is the discount factor and $\max_{a' \in \mathcal{A}} Q(s', a')$ is the maximum Q-value of the next state. Note that the maximum Q-value of a state is essentially equal to the value $V(s)$ (Lapan, 2018, p. 140).

Now we need to be able to calculate the loss in order to train a neural network. For a terminal state, the loss is calculated by subtracting the reward from the predicted Q-value and taking a square of that product (Lapan, 2018, p. 145).

$$\mathcal{L} = (Q_{s,a} - r)^2 \quad (2.2)$$

In a non-terminal state, the loss is calculated by subtracting the current reward, as well as the discounted maximum Q-value of the next state from the current Q-value prediction.

This allows us to "propagate" the reward from the next state and calculate Q-values for the preceding states (Lapan, 2018, p. 182).

$$\mathcal{L} = \left(Q_{s,a} - \left(r + \gamma \max_{a' \in \mathcal{A}} Q_{s',a'} \right) \right)^2 \quad (2.3)$$

Both of these losses are meant to be minimized.

In its pure form, this approach does not work well. Data obtained from sequential games contain correlation, which breaks the assumption of independent and identically distributed (*i.i.d.*) data points. This poses a serious problem for machine learning models (Goodfellow et al., 2016, p. 111). There are tricks that can help avoid this problem, and they are discussed in Section 4.5.1 together with practical implementation.

Moreover, because Q-learning makes choices based on the best available action, it can get trapped in a local maxima. According to Lapan, there is no single optimal way to balance exploration with exploitation for DQNs, as it is still an area of active research (Lapan, 2018, p. 146). To prevent an agent from getting stuck on a suboptimal solution, a hyperparameter ϵ is introduced, which is inspired by Sutton's formulation of a DQN agent. ϵ determines the probability of taking a random action instead of the one with the highest value. Over time, ϵ is decreased using linear or exponential decay until the policy stabilizes (Sutton & Barto, 2018, pp. 50-52).

2.3 Policy Gradients

Instead of calculating the value of a given state, policy gradient methods (PGs) aim to maintain good policy. A policy can be defined in terms of Q-values in the following way:

$$\pi(s) = \arg \max_a Q(s, a) \quad (2.4)$$

which means that the result of our policy π at every state s is the action with the largest Q-value (Lapan, 2018, p. 255).

However, unlike always aiming for the state with the highest Q-value, a neural network used in a PG samples a policy by converting the output logits of a neural network to probabilities. This introduces an element of randomness and allows them to perform better in stochastic environments (Lapan, 2018, p. 256). PGs start with a random policy that eventually becomes more deterministic. This approach implicitly balances exploration with exploitation without the need for explicit manipulation of hyperparameters (Lapan, 2018, p. 259).

2.3.1 REINFORCE Algorithm

A very simple implementation of a PG method is the REINFORCE algorithm. It initializes the policy network with random weights, plays N episodes and stores their transitions.

The total reward in episode k at step t is calculated as a sum of discounted rewards of all subsequent steps (Lapan, 2018, p. 259):

$$Q_{k,t} = \sum_{i=t}^T \gamma^{i-t} r_i \quad (2.5)$$

where γ^{i-t} is the discount factor and r_i is the reward for the step i (Sutton et al., 1999).

The loss function is calculated as following:

$$\mathcal{L} = - \sum_{k,t} Q_{k,t} \log(\pi(s_{k,t}, a_{k,t})) \quad (2.6)$$

where we multiply the log probability of the action taken $\pi(s_{k,t}, a_{k,t})$ with the discounted reward $Q_{k,t}$. We want to maximize the reward, but optimizers like standard gradient descent try to minimize the loss, so we have to use the $-$ sign (Lapan, 2018, p. 259). This calculation of state value puts it into the class of MC methods (see Section 1.2.1).

2.3.2 Actor-Critic Algorithm

The actor-critic method (A2C), is a policy gradient method that uses two neural networks. The actor network, also called the policy network, makes decisions that the agent should make by weighted sampling. The critic network, also called the value network, estimates the quality of these decisions, which is subtracted from the policy. The quality of decisions is estimated based on the difference between expectation and past performance. Over time, both the actor and the critic networks are updated. This architecture helps prevent the agent from becoming overconfident in their own choices and stabilizes learning (Lapan, 2018, p. 285). It can be represented with the following formula:

$$Q(s, a) = V(s) + A(s, a) \quad (2.7)$$

where $Q(s, a)$ is the total reward, $V(s)$ is the value of the current policy and $A(s, a)$ is the advantage of the chosen action a within the current state s (Lapan, 2018, p. 285).

2.4 Comparison

PG methods, like REINFORCE or A2C, are on-policy algorithms and require the training data to be based on the currently pursued policy. On the other hand, DQNs learn only from states and their transitions. DQNs try to make the most optimal choices, regardless of the situation, whereas PGs only improve the existing policy based on feedback from the environment. DQNs can train on old and irrelevant data and still improve, which can make training easier in certain circumstances. PGs require interaction with the environment because they have implicit assumptions about the optimality of the current policy (Lapan, 2018, p. 255).

Each have their own benefits and demerits, and their implementation, as well as findings of their usefulness for modeling Dominion, is explored in Chapter 4.

3. Deep Reinforcement Learning for the Card Game Dominion

This chapter explains the Dominion card game and its representation within deep reinforcement learning (deep RL). It then continues on to existing approaches for the game, both heuristic bots and previous attempts to train deep RL agents. Lastly, it compares agents to heuristic bots and quantifies their relative performance.

3.1 The Card Game Dominion in Reinforcement Learning

Dominion is a popular card game where the goal is to collect as many *victory cards* by the end of the game as possible. There is a *deck-building* mechanic - the player starts with a relatively weak deck and makes decisions that allow them to improve their deck over time. At the beginning of the game, players are dealt default cards into their decks and take turns playing until a condition triggers the end of the game. The condition may be triggered by an empty supply pile of *Province cards*, or three other empty supply piles of *action cards*. During a player's turn, there are the following phases (Vaccarino, 2016):

- The player draws 5 cards from their current deck
- If the player has any *action cards* in their hand, they can choose to use them
- If the player has any *coin cards*, they can choose to use them and purchase a card
- The player either purchases cards or abstains, then discards the cards in their hand and their turn ends
- If the player does not have any more cards in their deck, they shuffle their *discard pile* and draw from it

There are three main types of cards (Dominion Strategy Wiki, 2025c):

- *Victory cards* - these cards are essential for winning the game. The player with the most victory cards at the end of the game wins. However, these cards usually do not serve any other function and only take up valuable space in the deck.
- *Action cards* - these cards give the players useful actions, such as the ability to play multiple cards during the *action phase*, the ability to buy multiple cards during the *buy phase*, an increased buying power or some other special effect.
- *Coin cards* - these are necessary for buying new cards. The player starts with low-value *copper coins* and aims to obtain *gold coins* of higher value, which can be bought from the shop.

The variable element in the game comes from the randomized *kingdom*. A kingdom is a set of 10 selected action cards (Dominion Strategy Wiki, 2024c), of which there are 26 piles in

the base game (Dominion Strategy Wiki, 2025b). These are available in the shop during the game. By keeping the kingdom fixed, the reinforcement learning problem can be simplified.

To apply RL to Dominion, a representation of the state of the environment and the actions of the player must be made. A basic approach would involve modeling the state of the game as the numbers of cards in each pile, as well as the number of cards bought by the players (Yang & Kuo, 2019). A similar approach is described in Chapter 4. A more complex approach could include the cards that the player has currently in hand, bonuses from used action cards, or a custom measure of the quality of the deck (Fernandez & Anderson, 2020).

The player makes decisions during the action phase and the buying phase of their turn. The decisions in the action phase are quite complex - action cards can only be used under certain circumstances, and they sometimes involve gaining or discarding cards. The buying phase is relatively simple in comparison - a player can buy a card with the money they currently have in hand, or use bonuses from action cards that have been played during the action phase. The player can only buy multiple cards if they have used an action card that grants them additional buys. During the buying phase, the player decides to purchase one of the 17 cards in the store. Three of them are coin cards, four of them are victory cards and the remaining ten are action cards selected before the game, i.e. the kingdom. This makes modeling the buying phase quite simple compared to the action phase (Yang & Kuo, 2019).

3.2 Heuristics

Before diving into existing approaches, it is imperative to understand how heuristic bots work in Dominion. Below are the ones that are relevant for this thesis:

- The simplest heuristic bot is *Big Money*. This bot buys the Province card whenever possible, and the highest available coin card otherwise. It is relatively strong against inexperienced players (Dominion Strategy Wiki, 2024a).
- A modification of the Big Money bot is the *Smithy Bot*. It functions in the same way as Big Money, but it also buys a Smithy card, which allows it to draw additional cards. It is slightly stronger than Big Money (Dominion Strategy Wiki, 2024a).
- The *Double Witch* is a modification of the Big Money bot which also buys two Witch cards (Gerigk & Engels, 2023). These cards weaken the opponent by giving them curses (Dominion Strategy Wiki, 2025a).

Unlike heuristics, there also exist general strategies that require a deeper conceptual understanding of the game. They are as follows:

- The rush strategy focuses on hoarding cards as fast as possible. It is only available in certain kingdoms, when accumulating cards indiscriminately results in a victory points bonus. Kingdoms that contain the *gardens* card are well suited for this strategy (Dominion Strategy Wiki, 2022).

- Another strategy is the engine. The engine strategy aims to chain action cards in a way that draws out most of the player’s deck. This strategy allows the player to utilize their action cards to their full extent, and it can easily outperform the Big Money heuristic. It is a powerful strategy, but is also difficult to master (Dominion Strategy Wiki, 2024b).

3.3 Existing Approaches

In their paper, Yang and Kuo mention the specific difficulties of modeling Dominion with reinforcement learning. The first issue is the stochastic nature of the game because it involves shuffling cards. The second issue is the dynamic action set, where only some cards can be bought with the coins currently in hand. The authors decided to only model the buying phase of the game and they created simple heuristics for the action phase. A selection of heuristic bots, including Big Money, was used for training the agent, as well as self-play. The authors tested three different reward approaches while using the deep Q-network (DQN) as a base. A Monte Carlo (MC) modification of the reward system seemed to perform the best against heuristic bots, having superior performance compared to Q-values and SARSA. The authors also found that the neural network (NN) used in the DQN performs best with two hidden layers. The best agent in this paper manages to rival the performance of Smithy Bot, beating it around 83 percent of the time (Yang & Kuo, 2019).

Halawi et al. also chose to use DQNs for their approach and compared it to Smithy Bot, Double Witch bot and *Provincial* bot. Unlike the previously mentioned heuristic bots, Provincial is an artificial intelligence that uses evolutionary algorithms to develop a strategy that can outperform even skilled human players (Fisher, 2014). Similarly to the previously mentioned paper, the authors only modeled the buying phase of the game and left the action phase to heuristics. They trained their agent on over 7000 games of Dominion and achieved superior results. Their agent managed to win against Big Money 91 percent of the time and against Provincial 58 percent of the time (Halawi et al., 2024).

Like the previously mentioned papers, Fernandez and Anderson designed their agent using DQNs. The authors approached the problem as a Markov decision process (MDP) but conceded that the players do not have complete information about the game state. The structure of their NN was complex, but it essentially used four fully connected hidden layers before output. Instead of using Q-values, the authors decided to use SARSA for calculating rewards. To overcome the issues related to the sparse rewards, the authors used eligibility traces to assign credit from the end-game rewards. This modification bridges the gap between MC and TD methods by extending the number of steps that the reward is carried backward (Sutton & Barto, 2018). The authors did not mention how they prevented the neural network from making illegal buying or action decisions. Their trained agent was only able to beat a computer player that makes random decisions (Fernandez & Anderson, 2020).

Gerigk and Engles tried to create an agent that could handle all the complexities of Domin-

ion. The authors used most of the kingdoms available in the base game and created vector embeddings to represent the game state using a deep NN. The authors modeled both the action phase and the buying phase of the game, but they simplified the usage of action cards by iterating through all of the chained decisions as single card decisions. Instead of using DQNs, the authors chose to use a modified actor-critic (A2C) approach. The updates to the network during training were limited, thus making them "soft". The authors of the paper also added a temperature parameter to stabilize the training of their model. Overall their deep RL agent outperformed the Big Money bot, winning 73 percent of the simulated games. It struggled against the Double Witch bot, where it only won 42 percent of the games. When analyzing the choices made by their agent, the authors noted that it struggled against rush and engine strategies which require a different approach than Big Money strategies (Gerigk & Engels, 2023).

3.4 Comparison

Although it is difficult to compare the results of the various bots head-to-head, it seems as though the model developed by Halawi et al. performs the best against various heuristic strategies. Not only did it beat simple strategies most of the time, it also performed well against more complicated strategies or even bots playing at an advanced level (Halawi et al., 2024). From the results (see Table 3.1), it seems that beating simple heuristics is not very difficult (Fernandez & Anderson, 2020; Yang & Kuo, 2019), but any additional improvements to the heuristic bots make it more difficult to beat them (Gerigk & Engels, 2023; Halawi et al., 2024), where agents struggle to win against the Double Witch bot consistently.

Table 3.1

Comparison of the agent's winrate against various strategies. Data is extracted from relevant papers and converted to a decimal probability of winning against the bot. Source: Author.

	Big Money	Smithy	Double Witch	Provincial
(Yang & Kuo, 2019)	-	0.83	-	-
(Halawi et al., 2024)	0.91	0.85	0.56	0.58
(Fernandez & Anderson, 2020)	-	-	-	-
(Gerigk & Engels, 2023)	0.73	-	0.42	-

Table 3.2 compares different approaches used by the authors of the papers as well as modifications to the game representation. Three of the authors started with DQNs but adapted the reward structure to increase performance later. Deep RL approaches are described in Chapter 2. The general RL approaches, which inspired the reward structure, are explained in Section 1.2

Table 3.2

Comparison of the reinforcement learning approaches used in different papers. Abbreviations used: DQN - deep Q-network, PG - policy gradients, MC - Monte Carlo, TD - temporal-difference. Data is extracted from relevant papers. Source: Author.

	Approach	Game representation	Reward structure
(Yang & Kuo, 2019)	DQN	Buying phase	MC, SARSA, Q-learning
(Halawi et al., 2024)	DQN	Buying phase	Q-learning
(Fernandez & Anderson, 2020)	DQN	Complex	SARSA
(Gerigk & Engels, 2023)	PG	Complex	-

4. Design of Deep Reinforcement Learning approach

The papers mentioned in the previous chapter described the game of Dominion as a partially observable Markov decision process (Fernandez & Anderson, 2020), which contains elements of stochasticity (Yang & Kuo, 2019). In general, this game has the following characteristics:

- Every decision leads to a new game state until the game ends.
- Some choices are only available under specific conditions. For example, when buying cards, the player has to have coin cards of sufficient value in their hands.
- All used cards end up in the discard pile, they get shuffled back into the main deck. This adds an element of stochasticity.
- There is imperfect information. Since the player does not know the opponent's cards, the state representation is limited.

Traditional deep reinforcement learning methods (see Section 1.2), should still be applicable to the complex environment of this game, as shown by previous work in this area (see Section 3.2). However, some adjustments can be made to simplify the problem and achieve satisfactory results with the chosen reinforcement learning methods and limited computing power.

4.1 Thesis Goals and Research Questions

The goals of this thesis are to propose an approach using deep reinforcement learning for the card game Dominion (possibly its simplification), experiment with its variants, and compare it with other approaches.

In this context, the thesis addresses the following research questions (RQs):

1. Can a properly trained deep RL agent achieve and surpass the performance obtained by a simple heuristic in the card game Dominion?
 - To measure this quality, the win rate of the agent against a heuristic bot across one thousand games is used.
2. What are the best settings of a deep RL agent to perform better in a partially observable environment?
 - These adjustments can include the structure of the rewards, simplification of the game or the type of reinforcement architecture used by the agent.
3. Supposing that the agent performs at a satisfactory level, what impact do the specific choices made by a trained deep RL agent have in the card game Dominion?
 - The choices made by the trained agent are evaluated using the domain knowledge

of the author, to better understand the decisions that it makes and why it makes them.

4.2 Methodology

To achieve the goals and answer the research questions stated in Section 4.1, the following steps are taken:

1. An analysis of existing deep reinforcement learning approaches for the card game Dominion is undertaken (see Section 3.2)
2. A custom deep RL agent is designed with the goal of achieving superior performance.
3. The agent is implemented in Python and functionality is tested to ensure adherence to rules of the game.
4. The trained agent is compared to a Big Money bot. If it fails to match the bot's performance, steps 3 and 4 are repeated.
5. Benchmarks are selected for comparison to existing approaches and to evaluate relative strength.
6. The performance of the agent is quantified for numerical comparison with the secondary goal of also achieving qualitative understanding of its actions or policy.

4.3 Approach

There are some specifics to working with neural networks and a reinforcement learning environment, such as the representation and implementation of the game Dominion within the framework of Python. They are described in the following sections.

4.3.1 Game Representation

In reinforcement learning, the agent should make decisions given the observed state of the environment. In order for the choices to be applicable in the game, precautions are taken so that the state reflects the information that a normal player would take. The game state is represented as a vector with length of 34. Since there are 17 cards that can be bought, the player's deck represents the number of cards bought by the player. The number of cards that the opponent holds is implicitly mentioned as a difference between the cards in the supply and the player's hand. Adding additional information, such as the number of coins available to the player during the buying phase, was considered. However, it was left out as it would add noise and was redundant due to masking described in Section 4.3.2.

$$\text{state} = \begin{bmatrix} \mathbf{n}_{supply} \\ \mathbf{n}_{deck} \end{bmatrix}$$

The action chosen by the agent is represented as an integer between 1 and 18, as follows:

- 1 → copper
- 2 → silver
- 3 → gold
- ...
- 18 → nothing

In the real world, a player can only guess how good their decisions are. In this deep RL environment, rewards are derived from the results of the game. Waiting until the results are available would not be possible in a real game, but this method allows the agent to learn effectively. Once trained, the agent should make valid choices even without immediate rewards.

4.3.2 Agent Design Choices

The chosen RL approach uses neural networks, which necessitates some adjustments. Firstly, a fairly large number of samples has to be collected so that the network can be fitted. Secondly, the independence of the samples must be ensured so that the trained weights are not biased. Lastly, some form of masking must be applied to prevent the agent from choosing illegal actions, which can be effectively implemented with a boolean mask passed to the NN.

Several choices had to be made to implement a reinforcement learning agent for the game Dominion. Given the results of previous papers, deep Q-networks were chosen as the primary method. They are relatively simple to implement (see Section 2.2), and their sampling method, which is always selecting the best action in a given state, is simple to explain. Policy gradients were left in reserve because they can better deal with uncertainty due to weighted sampling (see Section 2.3). Moreover, they offer adjustments, such as the actor-critic method, which can improve stability while training. There is no best method for calculating the reward, so the reward structure was left to experimentation. In the end, the best performing agent is chosen as the optimal solution.

To decrease the complexity of the game, several choices were made. Namely:

- The kingdom was fixed to the 10 recommended starter cards by the rulebook (Vaccarino, 2016).
- Action phase is modeled using a simple heuristic, which is implemented with *if statements*. It is derived from the domain knowledge of the author and described in detail in Section 4.4.
- Any game longer than 50 turns terminates in a loss for the agent.

4.4 Implementation Details

The whole reinforcement learning environment was created in *Python* (Python Software Foundation, 2024). The game itself was simulated in the package *pyminion* (Slack, 2024). *Pyminion* offers a simple interface where *Dominion* can be simulated. All of the Source Code is documented in Appendix A, along with instructions on how to run it.

The main game loop (Source code 4.1) contains an instantiation of the *CustomGame* class. It is a wrapper of the *Game* class, which simulates a game until a winning condition is triggered, which also includes a data buffer for variables used in reinforcement learning. Within this game, bots make choices using predefined behavior.

Source code 4.1

Simplified code for running a game. Source: Author.

```

1  # Set up the game
2  bot1 = BigMoney()
3  bot2 = MyBot()
4
5  game = CustomGame(
6      players=[bot1, bot2],
7      expansions=[custom_set],
8      memory=memory
9  )
10 game.play()
```

Note that *MyBot* is the bot that contains the RL agent. *Expansions* and *memory* within *CustomGame* allow the user to select the kingdom and pass the memory buffer respectively. The memory buffer contains information like game state, action chosen by the agent and the reward among others.

In a game, the custom bot goes through several phases during their turn (Source code 4.2).

Source code 4.2

Simplified bot phases. Source: Author.

```

1  def take_turn(self, game: "Game"):
2      self.start_turn(game, is_extra_turn)
3      self.start_action_phase(game)
4      self.start_treasure_phase(game)
5
6      # Save game state before action
7      self.decider.set_current_state(self, game)
8      self.start_buy_phase(game)
9      self.decider.reset_state()
10
11     self.start_cleanup_phase(game)
12     self.end_turn(game)
```


The only modification from the original code is that the game state gets saved before the buying phase begins. The behavior of the agent during the action phase and the buying phase is modified in separate functions, *action_priority* and *buying_priority*, which are called by the bot through a decider. In short, action priority yields which action cards should be used given a heuristic while buying priority yields which cards should be bought given a decision by the neural network using the state of the game.

The action priority is established by a simple heuristic, which was determined through domain knowledge of the author and trial and error. It is as follows (see Source code 4.3):

1. Action cards that give the agent additional actions are used first.
2. Action cards that allow the agent to swap cards are used only if they can improve the player's deck. Swapping the victory cards for action cards is useful because they only become relevant at the end of the game.
3. Action cards that allow the agent to draw more cards are used last.

Source code 4.3

Simplified action priority. Source: Author.

```

1  def action_priority(self, player: "Player", game: "Game"):
2      while (player.state.actions > 0):
3          available_cards = [
4              pile.cards[0] for pile in game.supply.piles \
5                  if len(pile.cards)>0
6          ]
7          if (village in player.hand.cards):
8              yield village
9          elif (market in player.hand.cards):
10             yield market
11          elif (merchant in player.hand.cards):
12             yield merchant
13          elif (cellar in player.hand.cards) and (
14              (estate in player.hand.cards) \
15              or (duchy in player.hand.cards) \
16              or (province in player.hand.cards) \
17              or (curse in player.hand.cards)
18          ): # Discard victory cards
19             yield cellar
20          elif (mine in player.hand.cards) and (
21              (
22                  (copper in player.hand.cards) \
23                  and (silver in available_cards)
24              ) or (
25                  (silver in player.hand.cards) \
26                  and (gold in available_cards)
27              )
28          ): # Upgrade coin cards if possible
29             yield mine

```

```

30         elif (militia in player.hand.cards):
31             yield militia
32         elif (smithy in player.hand.cards):
33             yield smithy
34         # elif remodel in player.hand.cards:
35         #     yield remodel
36         # Remodel isn't used due to complexity
37         elif (workshop in player.hand.cards) and (
38             (smithy in available_cards) \
39             or (village in available_cards) \
40             or (silver in available_cards)
41         ): # Only get valuable cards
42             yield workshop
43         elif moat in player.hand.cards:
44             yield moat
45         else:
46             return iter([]) # No action card chosen

```

The buying priority relies on the neural network that the agent uses. See Source Code 4.4 for details.

1. A mask is created for valid buying decisions. This mask is a vector that contains 0 for invalid decisions and 1 for valid decisions.
2. The state, which was saved earlier via a call from the function *take_turn*, is converted to a tensor.
3. The action mask is also converted to a tensor and both are passed to the agent's network.
4. Once a decision is obtained, it is mapped to the card and passed to the game engine for buying.

Source code 4.4

Simplified buying priority. Source: Author.

```

1  def buy_priority(self, player: "Player", game: "Game"):
2      # Filter valid buying decisions
3      valid_actions = [1] * 18
4      cards = list(self.action_mapping.values())[:17]
5      for i in range(17):
6          if cards[i].base_cost.money > player.state.money:
7              valid_actions[i] = 0
8          elif len(game.supply.piles[i].cards)==0:
9              valid_actions[i] = 0
10
11     # Convert game state to tensor
12     state_tensor = torch.tensor(
13         self.state,
14         dtype=torch.float32
15     ).unsqueeze(0)
16

```

```

17     # Get valid action mask
18     valid_action_mask = torch.tensor([valid_actions])
19
20     # Obtain a decision by the neural network
21     action, _ = player.net.get_action(
22         state_tensor, player.g, valid_action_mask
23     )
24
25     # Set and save the selected action
26     self.action = action
27     player.memory.states.append(self.state)
28     player.memory.actions.append(self.action)
29
30     # Set return card
31     card = self.action_mapping[self.action+1]
32     if not card:
33         return iter([])
34     else:
35         return iter([card])

```

The reward obtained by the agent at the end of the game was calculated as seen in Source Code 4.5. Because the order of players is randomized for every game, the agent is selected using its index string. The code for calculating it is as follows:

Source code 4.5

End reward calculation. Source: Author.

```

1  # Calculate the end reward
2  i = [str(player) for player in self.players].index('my_bot')
3  self.player_vp = self.players[i].get_victory_points()
4  self.enemy_vp = self.players[1-i].get_victory_points()
5
6  good_cards_ratio = self.players[i].get_good_cards_ratio()
7  end_reward = (
8      self.player_vp - self.enemy_vp
9  ) / 50 + good_cards_ratio

```

The code for obtaining good card ratio in the agent's inventory is detailed in Source code 4.6. This was added to the reward calculation to allow the bot to improve its inventory even if it failed to obtain additional victory points during the game. The code works as follows:

Source code 4.6

Ratio of good cards calculation. Source: Author.

```

1  def get_good_cards_ratio(self):
2      all_cards = [card for card in self.get_all_cards()]
3      good_cards = [card for card in all_cards if card not in [
4          curse, copper, estate
5      ]] # These are judged as low value cards by the author

```

```
6     ratio = len(good_cards) / len(all_cards)
7     return ratio
```

The code for evaluating the performance of an agent is detailed in Source code 4.7. The heuristic bot and the agent are initialized. The bot requires a trained network to make decisions. The players and selected kingdom are passed to a custom game object, and a simulator imported from the `pyminion` package is started with a specified number of iterations. Once it finishes, the results are printed on the console.

Source code 4.7

Simplified arena simulation. Source: Author.

```
1  # Set up the game
2  bot1 = BigMoney()
3  bot2 = MyBot(net=net)
4  game = CustomGame(
5      players=[bot1, bot2],
6      expansions=[custom_set]
7  )
8
9  # Simulate a virtual arena
10 sim = Simulator(game, iterations=1000)
11 result = sim.run()
12 print(result)
```

4.5 Experiments and their results

Different deep RL approaches are attempted in this chapter. The code used by the agents is available in Appendix A, along with instructions on how to train and evaluate them.

The general process of obtaining training data for the agents looks like this:

1. An agent obtains the weights of the neural network in a simulated game. When making decisions using the NN, the following parameters are added to a memory buffer:
 - The current state of the environment represented as a pytorch tensor (Ansel et al., 2024), where the values are normalized between 0 and 1.
 - The action selected by the agent, which is an output of the neural network given the state input. It is represented as an integer.
 - The reward is null at this point, and is retroactively assigned based on the reward system used by the agent. Note that all of the approaches listed below use different reward structures. However, the general reward calculation uses the difference in victory points at the end of the game as set out at the end of Section 4.4.
 - If the agent is not in a terminal state, the next state is also saved.

- A boolean variable whether the game is finished at this point, used in TD reward calculation.
2. Once a game is finished, a random experience is sampled out of the game.
 3. Games are played until a batch of experiences is ready. The only exception is when DQNs use a replay buffer. The details for that are explained in Section 4.5.1.
 4. The network is trained using the batch and using a reward calculation that is relevant for that approach.

4.5.1 DQN Agent

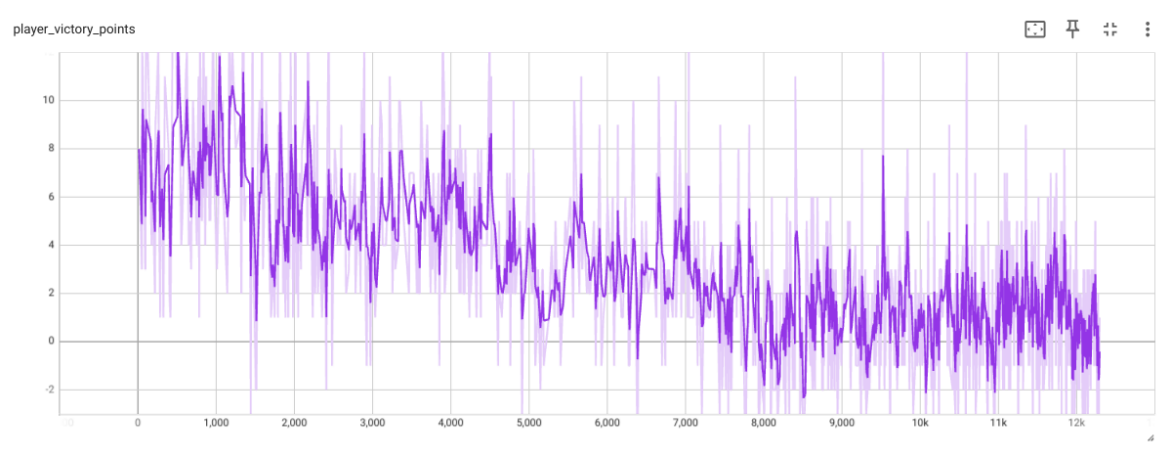
At first, deep Q-networks were chosen to make decisions for the agent. There were already successful implementations in the past (see Chapter 3.2) which made this a promising method.

Before training, the neural network used by the DQN was initialized. After that, Big Money was selected as the opponent for training. Games were simulated in parallel to fill an experience buffer, which was then passed to the DQN for training.

Lapan recommends adding a large replay buffer to stabilize training (Lapan, 2018, p. 147). An experience buffer with 10 thousand experiences was created and filled. A batch was sampled before starting training and data from new games replaced existing experiences. In total, around 500 thousand experiences were generated from 12 thousand games. Figure 4.1 shows the number of victory points received by the RL agent over time in a run that used the buffer. Because it did not improve training, the buffer was removed and experiences were sampled from individual games.

Figure 4.1

Change in the victory points received by the RL agent over time. The y-axis shows the number of victory points claimed at the end of the game and the x-axis shows the number of games simulated so far; source: Author.



After some modifications to the code, the games were simulated in parallel to speed up experience generation. To simplify the process, a training batch was created from single

experiences sampled from the games and was used for one training epoch. The network was trained over 100 thousand games.

The structure of the neural network used by the DQN agent was inspired by pytorch documentation (Paszke & Towers, 2024). It contained two hidden layers, each containing 128 neurons. ReLU activation was used between the hidden layers. It should be noted that the solution proposed by Halawi et al. and Fernandez and Anderson also used two hidden layers (Fernandez & Anderson, 2020; Halawi et al., 2024).

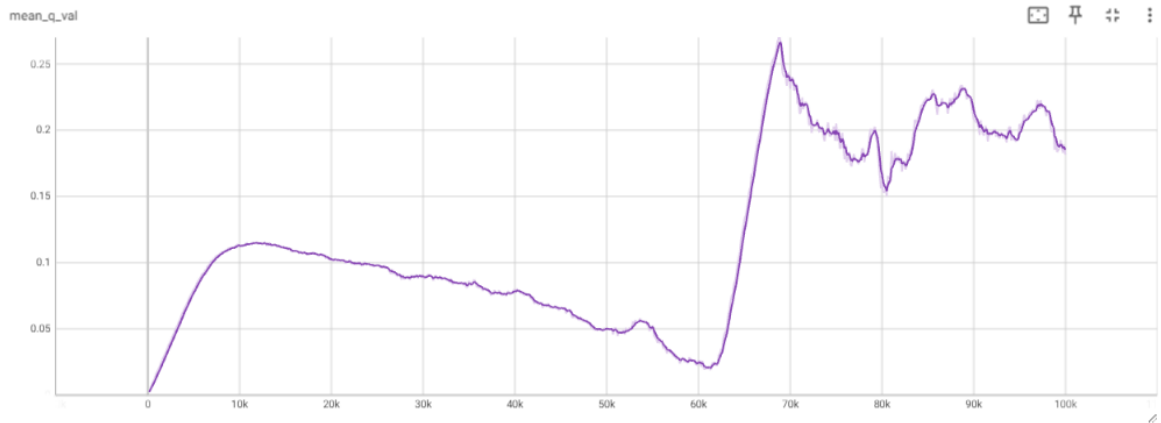
The hyperparameters used by the NN during training were as follows:

- Discount factor γ was set at 0.99.
- Learning rate was set at $1e - 4$
- The training batch contained 128 experiences.
- The epsilon started at 1 and decayed linearly to 0.1 over 80 thousand games.

This approach failed to yield an agent capable of beating Big Money. In a simulated arena where the DQN agent played against Big Money bot, it lost 99.2 percent of the games. However, the agent became very confident in its choices as seen from Figure 4.2. This figure displays the rise of Q-values over time. A higher Q-value means that the agent judges the state to be better.

Figure 4.2

Change in Q-values of states over time. The y-axis shows the mean Q-values of the states encountered during training and the x-axis shows the number of games simulated so far; source: Author.



There were numerous issues with this approach.

- There were a lot of hyperparameters that required tuning. There is no recommended way to decay ϵ , which presents issues while choosing a suitable decay schedule.
- The reward assigned to the agent was very sparse because Q-values only bootstrap the values of the following step.

- Not using a replay buffer reduced the advantage afforded by DQNs. As an off-policy method, they can take advantage of old data which reduces computational requirements (Lapan, 2018, p. 183). However, as the games were later parallelized, the advantages of off-policy methods over on-policy methods disappeared.

4.5.2 A2C Agent

Because the previous approach did not produce promising results, policy gradient methods were tested next. The actor-critic method was chosen because of its promising qualities, such as fast and stable convergence (see Section 2.3.2).

The overall approach was similar to that used in the previous section. A network was designed for the actor-critic agent. Big Money was selected as an opponent and games were simulated in parallel until a buffer was filled for training. After training, the process repeated until 100 thousand games were played.

The reward structure chosen was similar to the one used in Lapan’s book, where temporal-difference error was used. The network used by the A2C agent was also inspired by the one in Lapan’s book (Lapan, 2018, pp. 287-291). There was one hidden layer with 128 neurons and ReLU activation was used. Connected to this layer were two heads, the actor and the critic.

The hyperparameters used by the network during training were as follows:

- The discount factor γ was set at 0.99, same as the one used by Gerigk and Engels (Gerigk & Engels, 2023).
- The learning rate used by the network was set at $1e - 3$.
- The batch used for training contained 128 experiences. Each experience was sampled randomly from one game.
- To calculate the loss, the loss of the policy network was multiplied by 0.5 and the one used by the value network was multiplied by 0.01.
- When fitting the network, gradients were clipped at 1.0 to prevent exploding gradients.

This approach yielded an agent that was capable of beating a bot making random decisions. However, improvement stopped around 40 thousand steps and the agent was unable to consistently beat the heuristic bot Big Money. In an arena where 1000 games were simulated between the two, the A2C agent lost 72.2 percent of them

In the end, this architecture also proved to be suboptimal for the following reasons:

- Its performance failed to meet the benchmark set up by Big Money.
- Because the network has two heads, there are two additional hyperparameters that require tuning, each connected to the loss of the value or critic. This may have been responsible for the disappointing performance.

- Original A2C algorithm uses a reward structure similar to TD (see Section 1.2.2). This presents an issue of sparse rewards similar to the one in previous section.

Figure 4.3 shows number of victory points gained by the A2C agent when playing against Big Money bot. Figure 4.4 displays the number of victory points accumulated by Big Money bot. In general, the agent could never match the performance of the heuristic bot.

Figure 4.3

Victory points obtained by the agent over time. The y-axis shows the number of victory points gained by the agent at the end of the game and the x-axis shows the number of games simulated so far; source: Author.

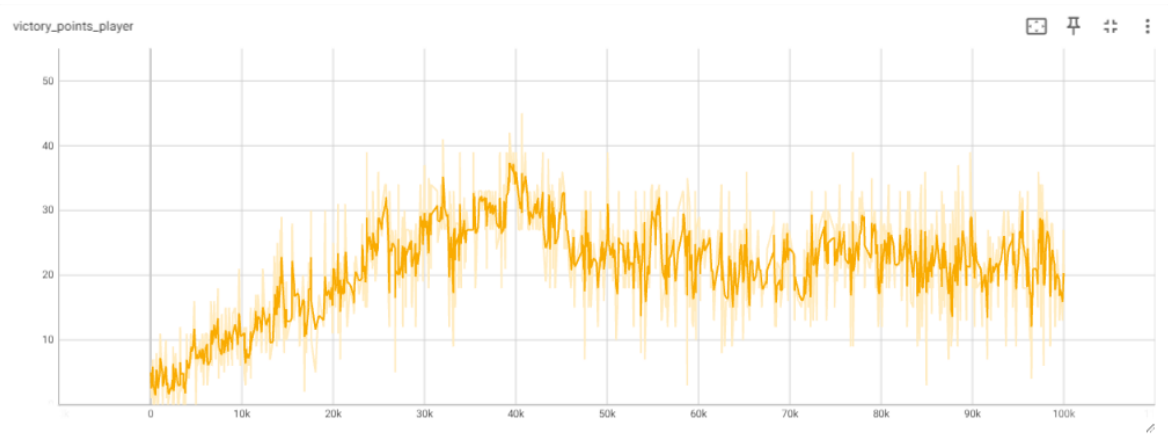
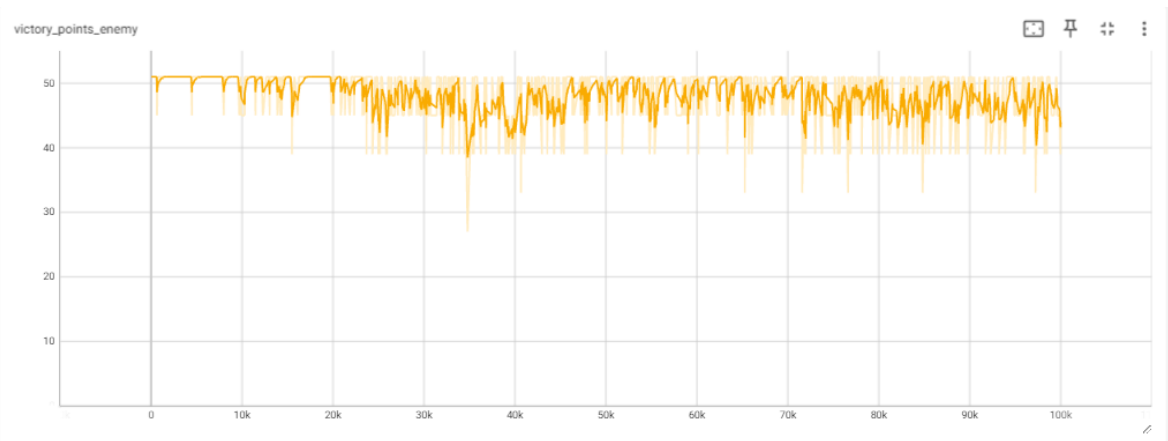


Figure 4.4

Victory points obtained by Big Money over time. The y-axis shows the number of victory points gained by the heuristic bot at the end of the game and the x-axis shows the number of games simulated so far; source: Author.



4.5.3 REINFORCE Agent

All of the approaches above dealt with issues related to hyperparameter optimization or sparse rewards. Since there were no more approaches left in reserve, the author decided to use a different approach to deal with the issues mentioned above.

REINFORCE was chosen as the main method for the following reasons:

- It only has one neural network that acts as the policy network. This reduces complexity compared to A2C architecture, which requires two networks, or at least two heads.
- It uses Monte Carlo for reward assignment. It was mentioned earlier that Dominion violates the Markov Property because the player does not have perfect information about the environment (Fernandez & Anderson, 2020). MC methods are better suited for environments that violate this property (Sutton & Barto, 2018, p. 137).
- Previous approaches suffered from sparse rewards as each experience contained rewards calculated using TD or Q-values respectively. MC sums all of the future rewards, which means that each turn is linked to the final reward. To prevent confusion, only one experience was sampled from each game and connected to the final reward without any discount parameter being used.

Firstly, a network was designed for the REINFORCE agent. It adapted the same network used by A2C in the previous section but removed the critic's head as it was no longer needed.

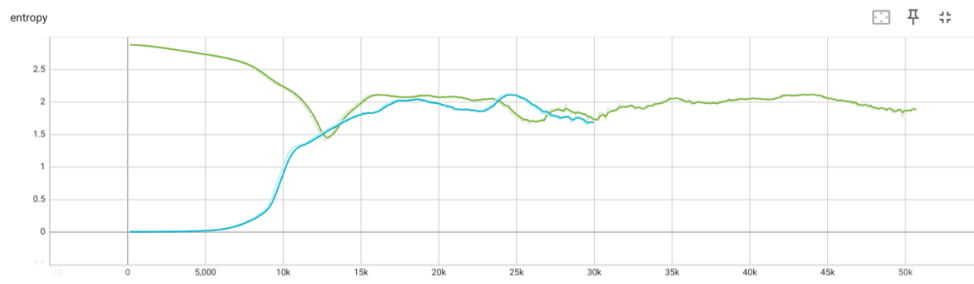
There was an experiment where the author modified the network with highly biased weights. Because a random policy has a high variance, a conservative policy that does not make any buying decisions should provide a more stable baseline. The network was initialized as follows:

- The bias for the last decision was set to 5, which was equal to not buying anything.
- The biases for all other decisions were set to -5 and later corrected through training.

Late testing revealed that this change was unnecessary. Figure 4.5 shows the entropy of a biased and unbiased network. At a certain point, they start following a similar learning path.

Figure 4.5

Entropy for the neural network for the biased and unbiased model. The y-axis contains the sum entropy of the output logits and the x-axis contains the number of games simulated so far. The green line shows the unbiased agent, blue line shows the biased agent; source: Author



Continuing on, Big Money bot was selected as the opponent, and games were simulated in parallel until a training batch was full. The training batch only contained the state, action and reward because information about the next state is redundant for MC. Same as before, 100 thousand games were simulated.

The only hyperparameter used by the network was:

- The gradients of the network were clipped to max 1.0 to ensure small updates.

There were no other hyperparameters that required tuning during training. It is worth noting that rewards were normalized during training using the following formula recommended by Lapan (Lapan, 2018, p. 464):

$$\tilde{r}_i = \frac{r_i - \mu_r}{\sigma_r + \delta}, \quad \delta = 10^{-8}. \quad (4.1)$$

where r_i is the original reward, μ_r is the mean, σ_r is the standard deviation, and δ is a constant to prevent division by zero. The normalized rewards had zero mean and unit variance. This modification may have played the agent converge to an optimal solution because it allowed the algorithm to differentiate between rewards with the same sign.

This approach yielded an agent that was able to rival the performance of Big Money bot after around 20 thousand simulated games. Beyond that, the bot did not improve significantly. When putting the REINFORCE agent in an arena that simulated 1000 games against Big Money bot, it won 94.5 percent of them. Because it fulfilled the goals of the thesis, there was no further experimentation with different models.

4.5.4 Summary

Three deep RL approaches were attempted: DQNs (Section 4.5.1), A2C (Section 4.5.2), and REINFORCE (Section 4.5.3). Table 4.1 shows the differences in architecture, training, and results of the different approaches.

Table 4.1

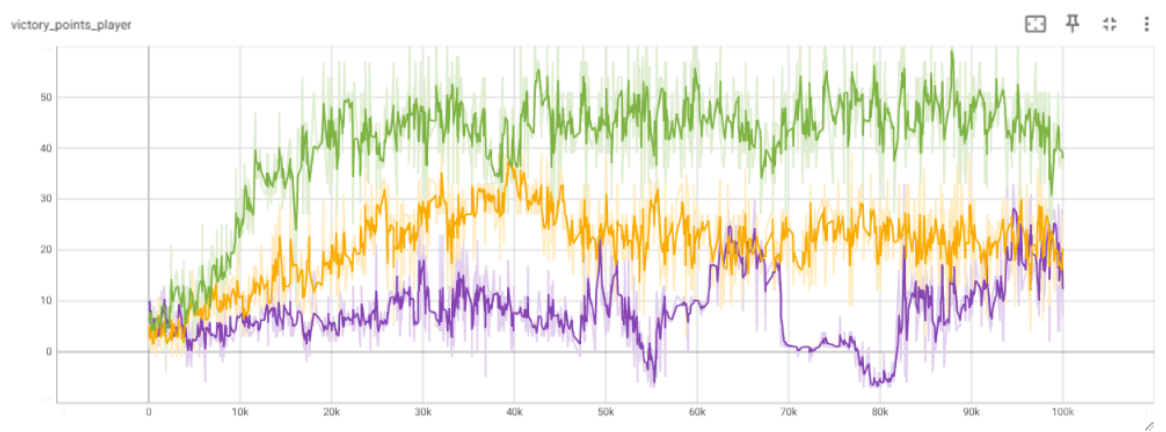
Comparison of differences between the agents. Explanation: Reward structure is connected to RL approaches in Section 1.2, Layers is the number of hidden layers used by the NN, Winrate is the percentage of games won against Big Money, Games is the number of simulated games until the agent converged. Source: Author.

	Reward structure	Layers	Winrate	Games
DQN (4.5.1)	Q-values	2	0	20k
A2C (4.5.2)	TD	1	0.17	40k
REINFORCE (4.5.3)	MC	1	0.95	-

Of these three, Both DQN and A2C dealt with issues connected to hyperparameter optimization and performed poorly, both during training and with subsequent tests. REINFORCE managed to achieve superior performance in an arena where it played against Big Money. The convergence dynamic of REINFORCE was superior to other agents. It is visible in Figure 4.6, which shows the number of victory points claimed at the end of each game during training. REINFORCE was the fastest to converge, attaining near-optimal policy after around 20 thousand simulated games. A2C converged after around 40 thousand games and slightly deteriorated after that. DQN never managed to converge, though there were occasional improvements in its performance. This could be explained by poorly optimized hyperparameters selected for the DQN agent.

Figure 4.6

Change in the victory points received by the RL agents over time. The y-axis shows the number of victory points claimed at the end of the game and the x-axis shows the number of games simulated so far. Green line represents REINFORCE agent, yellow represents A2C agent and purple represents DQN agent; source: Author.



It is also worth mentioning, that the rewards obtained by the REINFORCE agent seemed to contain much more information than the ones that used TD approaches, as is visible from Figure 4.7. Because rewards were only given at the end of the game, MC used by REINFORCE attributed strong rewards to all experiences. However, it is interesting that A2C agent managed to converge to a semi-optimal solution with rewards that were similar to those received by the DQN agent. This could be due to the on-policy nature of A2C, which allows it to converge with less training data (Sutton & Barto, 2018, p. 103).

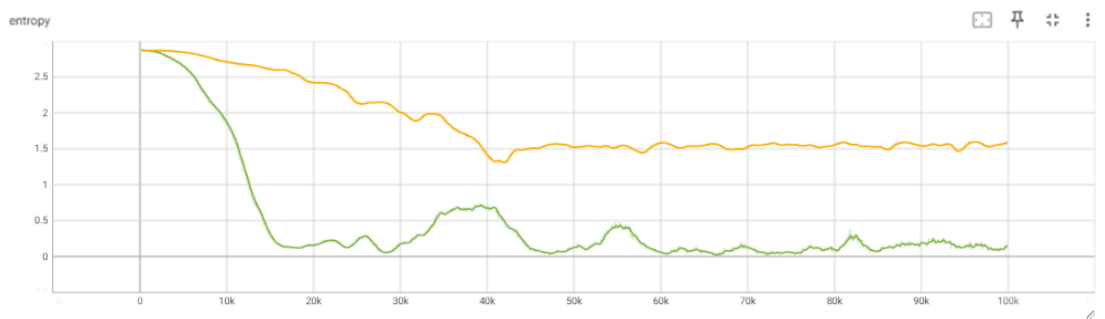
Lastly, the REINFORCE agent became much more confident in their policy compared to the A2C agent. It is visible in Figure 4.8, where the entropy of the selected agent was much lower for REINFORCE. This could be due to the comparatively dense rewards achieved by the agent or because the A2C agent was less confident in its choices due to the critic network.

Figure 4.7

Reward obtained by the agents during the training. The y-axis contains the reward obtained by the agent at the end of the game and the x-axis contains the number of games simulated so far. The green line represents REINFORCE agent, yellow line represents A2C agent and purple line represents DQN agent; source: Author

**Figure 4.8**

Entropy during training. The y-axis contains the entropy of the action chosen by the agent during training and the x-axis contains the number of games simulated so far. The yellow line represents A2C agent and the green line represents REINFORCE agent; source: Author



Despite receiving the same number of training experiences, REINFORCE delivered superior performance. Since the games were simulated in parallel, off-policy algorithm DQN did not provide any advantage over on-policy algorithms A2C and REINFORCE.

To answer the first research question presented in Section 4.1, a well trained REINFORCE agent surpassed the performance obtained by the Big Money bot and the Smithy bot after around 20 thousand experiences (RQ1), which corresponds to the same number of games. The winrate against Big Money bot is 95 percent and against Smithy bot is 88 percent.

Continuing on, from the experiments conducted in this thesis, it seems that MC methods perform better than TD methods, possibly due to the higher density of rewards in the training data. In addition, simplifying the environment by creating heuristics for the action phase and removing unnecessary hyperparameters for tuning helps achieve consistent performance by the agent (RQ2).

Lastly, the choices made by REINFORCE agent are evaluated using domain knowledge of the author. To achieve this goal, a single game is simulated against Big Money, which is available in Appendix A. The agent starts the game very aggressively by buying *militia* cards, which give the agent bonus coins and force the Big Money bot to discard two cards currently in its hands, which severely limits its buying power. When it is not possible for the agent to buy militia cards, it buys villages, which allow it to obtain more actions per turn. This allows the agent chain militia cards to obtain further coin bonuses. The agent uses the bonuses to buy gold coins. These are high value cards, that are used for buying provinces. In the end, REINFORCE agent ends up with 4 more provinces in its deck than the Big Money bot. When it is not possible for the agent to buy its preferred cards, it usually skips the turn buying nothing. This is good because it avoids buying low-value cards such as copper or curse, which decrease the overall strength of the deck. The strategy used by the agent can be classified as an engine strategy (Dominion Strategy Wiki, 2024b) and is judged as strong by the author (RQ3).

4.6 Comparison To Other Approaches

The three approaches presented in Section 4.5 were put in an arena where 1000 games were simulated against Big Money and Smithy bot respectively. There was no simulation against the Double Witch bot because the selected kingdom did not contain the Witch card.

Both the DQN and the A2C failed to even approach the performance of the methods mentioned in the papers in Section 3.2. However, the REINFORCE agent surpassed all existing papers when it came to performance against both the Big Money bot and the Smithy bot, as is visible in Table 4.2, which compares the performance of the trained agents against selected agents from Section 3.2. Due to the simplifications of the game, such as selecting a fixed kingdom or adding a heuristic for the action phase, the agent learned to play really well in this specific environment.

Table 4.2

Comparison of the agent’s winrate against various strategies. Data is from running 1000 games against the strategy or extracted from relevant papers (where applicable). The probabilities have been converted into a numerical format. Source: Author.

	Big Money	Smithy
DQN (4.5.1)	0.02	-
A2C (4.5.2)	0.17	0.08
REINFORCE (4.5.3)	0.95	0.88
(Yang & Kuo, 2019)	-	0.83
(Halawi et al., 2024)	0.91	0.85
(Gerigk & Engels, 2023)	0.73	-

Conclusion

This thesis started out by comparing existing deep reinforcement learning approaches for the card game Dominion. Based on the existing research, three approaches were implemented and evaluated: the DQN agent (Section 4.5.1), the A2C agent (Section 4.5.2) and the REINFORCE agent (Section 4.5.3). Several modifications were eventually added to the game, such as fixing the cards used in the kingdom and modelling the action phase of the game using a simple heuristic. These modifications simplified the game environment to the extent that the agent was able to successfully learn and develop a stable policy.

Existing papers achieved good performance with agents using DQN architecture, so it was selected as the main approach. Several modifications were added to the base architecture, such as enabling parallel game execution with multiprocessing in Python (Python Software Foundation, 2024). Despite experiments, such as the addition of a large experience buffer for sampling experiences, recommended by Lapan (Lapan, 2018, p. 174), the DQN agent did not perform at a satisfactory level and was later discarded.

There was one paper in the selected sources that relied on policy gradients (Gerigk & Engels, 2023), using an A2C architecture. The author implemented an agent inspired by this paper in Python and trained it. Although it achieved slightly better performance, there were still problems related to hyperparameter optimization and sparse rewards, so it was also dropped.

After the failures of the previous approaches, the author decided to use a REINFORCE agent with Monte Carlo rewards (see Section 4.5.3). This agent started to improve rapidly and achieved superior performance against the Big Money bot, winning 95 percent of the simulated games against it. The agent performed better than all of the existing solutions, although it should be noted that some of them use slightly more complex representation of the game. This approach offers major benefits. Because it has very few hyperparameters that would require optimization, convergence is much more likely. Moreover, it can learn in an environment where rewards are only assigned at the end of the game, so that the user does not have to create custom rewards using domain knowledge of the game.

Based on the experiments conducted, the author concludes that a properly trained deep RL agent can surpass the performance obtained by a simple heuristic in the Dominion card game after about 20 thousand simulated games. A deep RL agent can perform better in a partially observable environment by using a RL method that avoids sparse rewards, such as MC (see Section 1.2.1), and by decreasing the complexity of the problem through ways such as creating heuristics or simplifying models into ones that require fewer hyperparameters (see Section 4.5.4). Examining the choices made by the agent using domain knowledge reveals that it built a fairly strong engine strategy (see Section 3.2), which would most likely challenge a skilled player.

Future research should focus on whether REINFORCE still provides superior performance

on a more complex problem, such as when the kingdom is randomized or when the approach extends to the action phase of the game as well. The REINFORCE approach should also be tested in an environment where action cards are embedded, as proposed by Gerigk and Engels (Gerigk & Engels, 2023).

References

- Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., ... Chintala, S. (April 2024). PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. <https://doi.org/10.1145/3620665.3640366>
- Bellman, R. (1966). Dynamic programming. *Science*, 153(3731), 34–37. <https://doi.org/10.1126/science.153.3731.34>
- Chapman, A. C., Leslie, D. S., Rogers, A., & Jennings, N. R. (2013). Convergent learning algorithms for unknown reward games. *SIAM Journal on Control and Optimization*, 51(4), 3154–3180. <https://doi.org/10.1137/120893501>
- Darwin, C., & Huxley, J. (2003). *The origin of species: By means of natural selection of the preservation of favoured races in the struggle for life*. Signet Classics.
- Dominion Strategy Wiki. (December 2022). Rush (strategy) [Last edited December 19, 2022. Accessed April 7, 2025]. [https://wiki.dominionstrategy.com/index.php/Rush_\(strategy\)](https://wiki.dominionstrategy.com/index.php/Rush_(strategy))
- Dominion Strategy Wiki. (August 2024a). Big money [Last edited August 28, 2024. Accessed April 7, 2025]. https://wiki.dominionstrategy.com/index.php/Money_strategies
- Dominion Strategy Wiki. (September 2024b). Engine [Last edited September 2, 2024. Accessed April 7, 2025]. <https://wiki.dominionstrategy.com/index.php/Engine>
- Dominion Strategy Wiki. (December 2024c). Kingdom [Last edited December 23, 2024. Accessed April 7, 2025]. <http://wiki.dominionstrategy.com/index.php?title=Kingdom&oldid=89594>
- Dominion Strategy Wiki. (January 2025a). Witch [Last edited January 11, 2025. Accessed April 7, 2025]. <https://wiki.dominionstrategy.com/index.php/Witch>
- Dominion Strategy Wiki. (March 2025b). Dominion (base set) [Last edited March 5, 2025. Accessed April 7, 2025]. https://wiki.dominionstrategy.com/index.php/Card_types
- Dominion Strategy Wiki. (April 2025c). Card types [Last edited April 1, 2025. Accessed April 7, 2025]. https://wiki.dominionstrategy.com/index.php/Card_types
- Farama Foundation. (2024). Basic usage — Gymnasium documentation [Accessed March 31, 2025]. https://gymnasium.farama.org/introduction/basic_usage/
- Fernandez, G., & Anderson, B. (2020). Playing "dominion" with deep reinforcement learning [CS238 Final Project – Autumn 2020, Stanford University]. <https://benanderson.work/assets/238-final-report.pdf>
- Fisher, M. D. (2014). Provincial: A kingdom-adaptive ai for dominion [Unpublished technical report, Stanford University. Accessed: 2025-05-11]. <https://graphics.stanford.edu/~mdfisher/DominionAI.html>

-
- Gerigk, J., & Engels, S. (October 2023). Playing various strategies in dominion with deep reinforcement learning. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 19(1), 224–232. <https://doi.org/10.1609/aiide.v19i1.27518>
- Geva, S., & Sitte, J. (1993). A cartpole experiment benchmark for trainable controllers. *IEEE Control Systems Magazine*, 13(5), 40–51. <https://doi.org/10.1109/37.236324>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <http://www.deeplearningbook.org>
- Halawi, D., Sarmasi, A., Saltzen, S., & McCoy, J. (2024). Dominion: A new frontier for ai research. <https://arxiv.org/abs/2405.06846>
- Ho, J., Huang, J., Chang, B., Liu, A., & Liu, Z. (2023). Reinforcement learning: Playing tic-tac-toe [Preprint]. <https://www.techrxiv.org/doi/full/10.36227/techrxiv.20407575>
- Kolomazník, V. (2025). Code for deep reinforcement learning for the card game dominion [GitHub repository]. <https://github.com/Yes-and/bachelors-thesis>
- Lapan, M. (2018). *Deep reinforcement learning hands-on: Apply modern rl methods, with deep q-networks, value iteration, policy gradients, trpo, alphago zero and more*. Packt Publishing. <https://books.google.cz/books?id=xKdhDwAAQBAJ>
- Maillard, O.-A., Ryabko, D., & Munos, R. (2011). Selecting the state-representation in reinforcement learning. *Advances in Neural Information Processing Systems*, 24.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (December 2013). Playing atari with deep reinforcement learning [arXiv preprint]. <https://arxiv.org/abs/1312.5602>
- Mousavi, S. S., Schukat, M., & Howley, E. (2018). Deep reinforcement learning: An overview. In Y. Bi, S. Kapoor, & R. Bhatia (Eds.), *Proceedings of sai intelligent systems conference (intellisys) 2016* (pp. 426–440). Springer International Publishing. https://doi.org/10.1007/978-3-319-56991-8_32
- Paszke, A., & Towers, M. (2024). Reinforcement learning (dqn) tutorial [Last updated June 18, 2024. Originally published March 24, 2017]. *PyTorch*. https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html
- Python Software Foundation. (2024). *Python language reference, version 3.12*. <https://www.python.org>
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., & Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419), 1140–1144. <https://doi.org/10.1126/science.aar6404>
- Slack, E. (2024). Pyminion: Dominion game engine and simulator [Version 0.5.0, released on September 3, 2024]. <https://github.com/evanofslack/pyminion>
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. <https://mitpress.mit.edu/9780262039246/reinforcement-learning/>
- Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (1999). Policy gradient methods for reinforcement learning with function approximation. In S. A. Solla, T. K. Leen, &
-

- K.-R. Müller (Eds.), *Advances in neural information processing systems* (pp. 1057–1063, Vol. 12). MIT Press. https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf
- Tokic, M. (2010). Adaptive ϵ -greedy exploration in reinforcement learning based on value differences. In R. Dillmann, J. Beyerer, U. D. Hanebeck, & T. Schultz (Eds.), *Ki 2010: Advances in artificial intelligence* (pp. 203–210). Springer. https://doi.org/10.1007/978-3-642-16111-7_23
- Vaccarino, D. X. (2016). *Dominion (2nd ed.) [rulebook]* [Accessed April 7, 2025]. Rio Grande Games. <https://www.riograndegames.com/wp-content/uploads/2016/09/Dominion2E.pdf>
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3), 229–256. <https://doi.org/10.1007/BF00992696>
- Yang, H.-I., & Kuo, Y.-C. (2019). An AI for Dominion using deep reinforcement learning [Unpublished manuscript]. <https://api.semanticscholar.org/CorpusID:215776227>
- Zanotti, C., Rotiroti, M., Sterlacchini, S., Cappellini, G., Fumagalli, L., Stefania, G. A., Nannucci, M. S., Leoni, B., & Bonomi, T. (2019). Choosing between linear and nonlinear models and avoiding overfitting for short and long term groundwater level forecasting in a linear system. *Journal of Hydrology*, 578, 124015. <https://doi.org/10.1016/j.jhydrol.2019.124015>

Appendices

A. Source Code

All of the source code is available on an online repository under the following link <https://github.com/Yes-and/bachelors-thesis> (Kolomazník, 2025).

The repository contains all the files that were used during the training and evaluation of deep reinforcement learning agents.

Files *README.md* contains instructions for using the repository. In order to run the code, it is necessary to create a virtual environment with packages listed in *requirements.txt*

Files *test_DQN.py*, *test_A2C.py* and *test_REINFORCE.py* are used for testing the performance of existing agents. They print out the results into the console. The results of previous runs are available in the folder *selected_tests*. The weights of the neural networks are saved in the folder *selected_models* under their respective names. The REINFORCE agent also has a test file for evaluating individual agent choices during the game, which is located in the file *evaluate_REINFORCE.py*. The output of its run is located in the *selected_tests* folder. Section 4.5.4 refers to this run.

Files *train_DQN.py*, *train_A2C.py* and *train_REINFORCE.py* contain scripts for training the agents. A class defining the hyperparameters to be used is defined at the top of each file. Note that the training files use the *multiprocessing* library for parallel game simulation, which is a standard part of Python (Python Software Foundation, 2024). Each of the training files saves important information related to training and weights of the trained models periodically.

Folder *game_logic* contains files that modify the behavior of a simulated game from the *pyminion* package (Slack, 2024). File *my_bot.py* in the *bots* folder defines the behavior of the deep RL agent and its choices within the game. File *data_structures.py* defines a custom memory buffer used to collected experiences from the games. File *game.py* modifies the behavior of the game simulation from the *pyminion* package for deep RL. File *sets.py* defines the kingdom of cards used during the games.

Folder *src* contains additional files that are used during the game simulation for the deep RL agents. File *buffers.py* defines a multiprocessing-safe buffer, which is used for storing experiences for training. File *logger.py* defines a logger which was used for debugging during game simulation. File *networks.py* defines the bodies of all networks used by the deep RL agents. It uses the *pytorch* library (Ansel et al., 2024).

Folder *training* contains files that are used for game simulation and training the agent's network once a batch is prepared. File *simulate_game.py* defines how each agent simulates a game and returns experiences to the main training file *train_X.py*. File *train_network.py* contains the reward calculation for each deep RL agent and training of the network using a batch of experiences.